



Chương 7

QUẢN LÝ BỘ NHỚ

NỘI DUNG

- **Khái niệm cơ sở**
- **Các kiểu địa chỉ nhớ**
- **Chuyển đổi địa chỉ**
- **Overlay và swapping**
- **Mô hình quản lý bộ nhớ đơn giản**
 - **Cơ chế phân đoạn (Segmentation)**
 - **Cơ chế phân trang (Paging)**
 - **Cấu trúc bảng phân trang**
 - **Kiến trúc Intel 32bit và 64bit**
 - **Kiến trúc ARM**

MỤC TIÊU

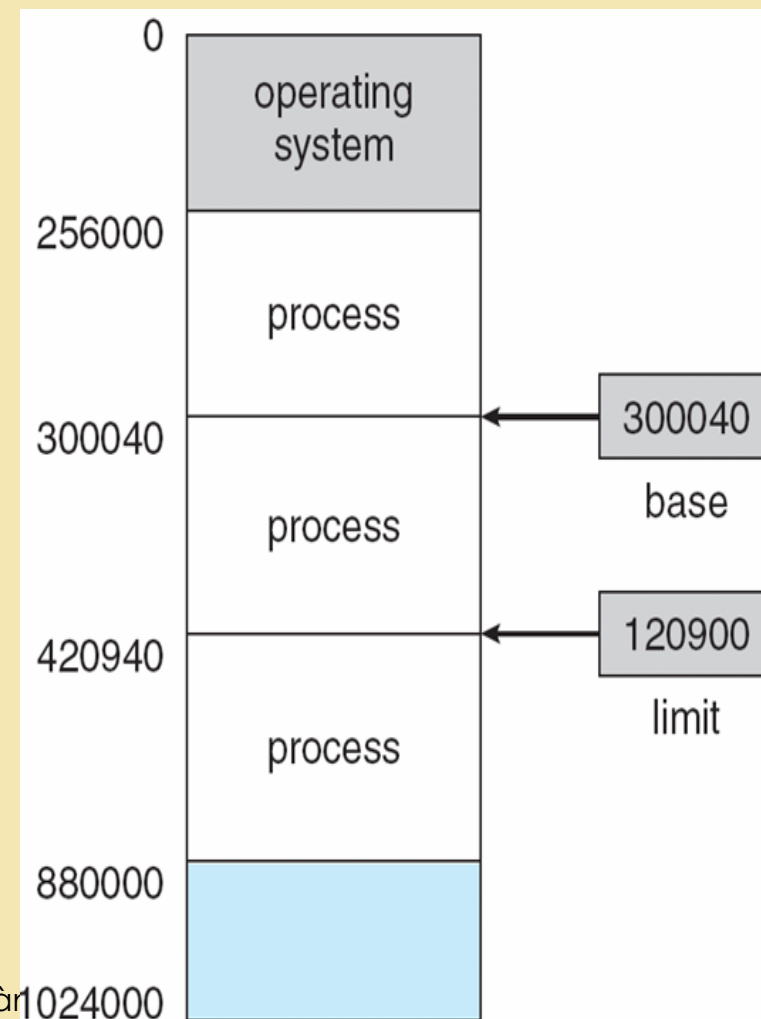
- ✓ Mô tả chi tiết các cách tổ chức khác nhau của bộ nhớ vật lý
- ✓ Thảo luận các phương pháp quản lý bộ nhớ khác nhau, bao gồm phương pháp phân trang và phân đoạn
- ✓ Mô tả chi tiết kiến trúc Intel Pentium, một kiến trúc hỗ trợ cả cách quản lý bộ nhớ: phân đoạn thuần túy và phân đoạn kết hợp phân trang

Khái niệm cơ sở

- Chương trình phải được mang vào trong bộ nhớ và đặt nó trong một tiến trình để chạy để được xử lý.
- **Input Queue** – tập hợp của những tiến trình trên đĩa mà đang chờ để được mang vào trong bộ nhớ để thực thi.
- Bộ nhớ chính và các thanh ghi là bộ lưu trữ duy nhất mà CPU có thể truy cập trực tiếp
- Đơn vị nhớ chỉ thấy dòng địa chỉ trong bộ nhớ
- Thanh ghi truy cập trong 1 chu kỳ đồng hồ của CPU (hoặc ít hơn)
- Bộ nhớ chính tốn nhiều chu kỳ đồng hồ, tạo ra một **stall**
- **Cache** được đặt giữa bộ nhớ chính và các thanh ghi CPU
- Việc bảo vệ bộ nhớ đòi hỏi phải đảm bảo các thao tác chính xác

Khái niệm cơ sở

- Quản lý bộ nhớ là công việc của HĐH với sự hỗ trợ của phần cứng nhằm phân phối, sắp xếp hiệu quả các tiến trình trong bộ nhớ.
- Mục tiêu cần đạt được là nạp càng nhiều tiến trình vào bộ nhớ càng tốt (gia tăng mức độ đa chương) .
- Trong hầu hết các hệ thống, kernel sẽ chiếm một phần cố định bộ nhớ; phần còn lại phân phối cho các tiến trình.



Khái niệm cơ sở

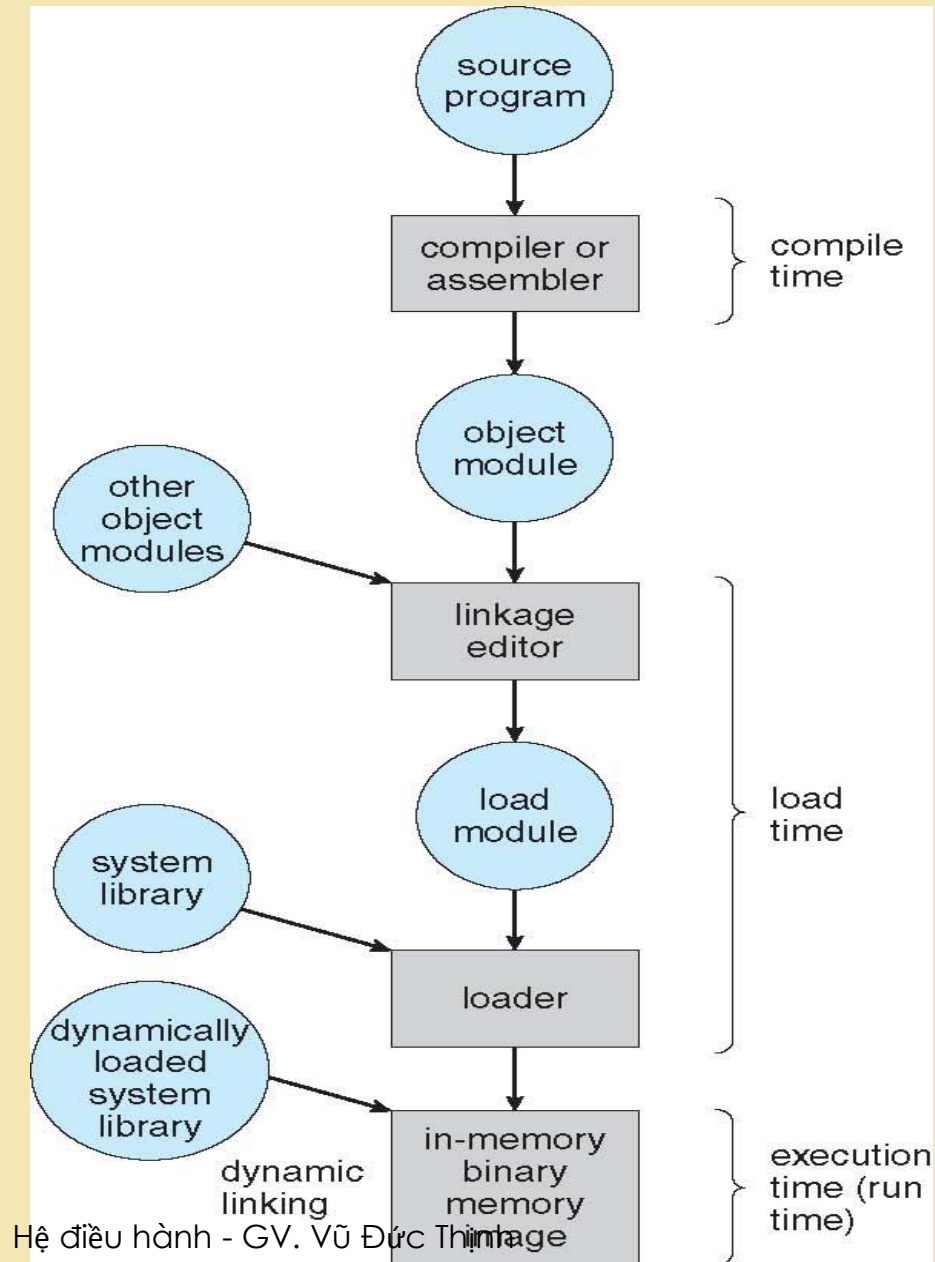
- Các yêu cầu đối với việc quản lý bộ nhớ :
 - Cấp phát bộ nhớ cho các tiến trình
 - Tái định vị (relocation)
 - Bảo vệ: phải kiểm tra truy xuất bộ nhớ có hợp lệ không
 - Chia sẻ: cho phép các tiến trình chia sẻ vùng nhớ chung
 - Kết gán địa chỉ nhớ luận lý của user vào địa chỉ thực

Các kiểu địa chỉ nhớ

- Địa chỉ vật lý (physical address)(địa chỉ thực) là một vị trí thực trong bộ nhớ chính
- Địa chỉ luận lý (logical address) là một vị trí nhớ được diễn tả trong 1 chương trình :
 - Địa chỉ tương đối (relative address): là một kiểu địa chỉ luận lý trong đó các địa chỉ được biểu diễn tương đối so với một vị trí xác định nào đó trong chương trình.
 - Địa chỉ tuyệt đối (absolute address): địa chỉ tương đương với địa chỉ thực.

Các bước xử lý của một chương trình người dùng

- Bộ linker: kết hợp các object Module thành 1 file nhị phân khả thực thi gọi là Load Module.
- Bộ loader: nạp Load Module vào bộ nhớ chính.



Liên kết động (Dynamic Linking)

- Quá trình link đến một **module ngoài** (external module) được thực hiện sau khi đã tạo xong load module (i.e. file có thể thực thi, executable).
- Load module chứa các **stub** tham chiếu (refer) đến routine của external module.
 - Lúc thực thi, khi stub được thực thi lần đầu (do tiến trình gọi routine lần đầu), stub nạp routine vào bộ nhớ, tự thay thế bằng địa chỉ của routine và routine được thực thi.
 - Các lần gọi routine sau sẽ xảy ra bình thường.
- Stub cần sự hỗ trợ của OS (như kiểm tra xem routine đã được nạp vào bộ nhớ chưa).

Ưu điểm của liên kết động

- Thông thường, external module là một thư viện cung cấp các tiện ích của OS. Các chương trình thực thi có thể dùng các phiên bản khác nhau của external module mà **không cần** sửa đổi, biên dịch lại.
- **Chia sẻ mã** (code sharing): một external module chỉ cần nạp vào bộ nhớ một lần. Các process cần dùng external module này thì cùng chia sẻ đoạn mã của external module → tiết kiệm được không gian nhớ và đĩa.
- Phương pháp dynamic linking cần sự hỗ trợ của OS trong việc kiểm tra xem một thủ tục nào đó có thể được chia sẻ giữa các tiến trình hay là phần mã của riêng một tiến trình (bởi vì chỉ có OS mới có quyền thực hiện việc kiểm tra này).

Nạp động (Dynamic loading)

- Cơ chế: chỉ khi nào cần được gọi đến thì một thủ tục mới được nạp vào bộ nhớ chính → tăng độ hiệu dụng của bộ nhớ (memory utilization) bởi vì các thủ tục không được gọi đến sẽ không chiếm chỗ trong bộ nhớ.
- Rất hiệu quả trong trường hợp tồn tại khối lượng lớn mã chương trình có tần suất **sử dụng thấp**, không được sử dụng thường xuyên (ví dụ các thủ tục xử lý lỗi)
- Hỗ trợ từ OS:
 - Thông thường, user chịu trách nhiệm thiết kế và hiện thực các chương trình có dynamic loading.
 - OS chủ yếu cung cấp một số thủ tục thư viện hỗ trợ, tạo điều kiện dễ dàng hơn cho lập trình viên.

Cơ chế phủ lấp (overlay)

- Tại mỗi thời điểm, chỉ giữ lại trong bộ nhớ những lệnh hoặc dữ liệu cần thiết, giải phóng các lệnh/dữ liệu chưa hoặc không cần dùng đến.
- Cơ chế này rất hữu dụng khi kích thước một tiến trình lớn hơn không gian bộ nhớ cấp cho tiến trình đó.
- Cơ chế này được điều khiển bởi người sử dụng (thông qua sự hỗ trợ của các thư viện lập trình) chứ không cần sự hỗ trợ của OS

Ví dụ về Overlay

- Cho P 1 = 70 KB
P 2 = 80 KB
Symbol table = 20 KB
Common routines = 30 KB
- Total memory available = 150 KB

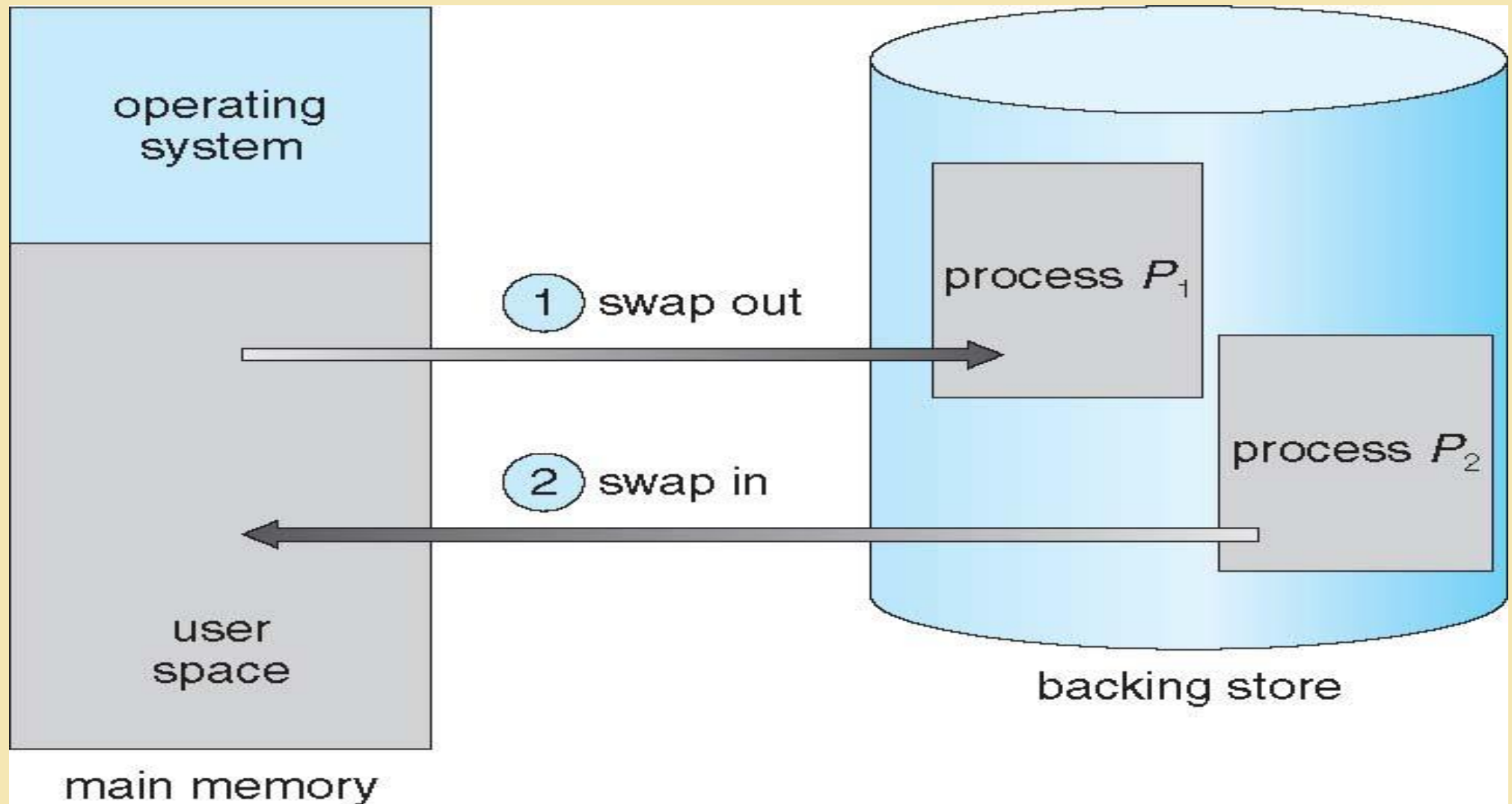
Cơ chế hoán vị (swapping)

- Một tiến trình có thể tạm thời bị swap ra khỏi bộ nhớ chính và lưu trên một hệ thống lưu trữ phụ. Sau đó, tiến trình có thể được nạp lại vào bộ nhớ để tiếp tục quá trình thực thi.

Swapping policy:

- Round-robin: swap out P1 (vừa tiêu thụ hết quantum của nó), swap in P2, thực thi P3...
- Roll out, roll in: dùng trong cơ chế định thời theo độ ưu tiên (priority-based scheduling): tiến trình có độ ưu tiên thấp hơn sẽ bị swap out để nhường chỗ cho tiến trình có độ ưu tiên cao hơn mới đến được nạp vào bộ nhớ để thực thi

Minh họa cơ chế Swapping



Chuyển đổi địa chỉ

- Chuyển đổi địa chỉ: là quá trình ánh xạ một địa chỉ từ không gian địa chỉ này sang không gian địa chỉ khác.
- Biểu diễn địa chỉ nhớ:
 - Trong source code: symbolic (các biến, hằng, pointer..)
 - Thời điểm biên dịch: Thường là địa chỉ khả tái định vị
 - Thời điểm linking/loading: có thể là địa chỉ thực.

Chuyển đổi địa chỉ (tt)

- Địa chỉ lệnh (instruction) và dữ liệu (data) được chuyển đổi thành địa chỉ thực có thể xảy ra tại 3 thời điểm khác nhau
 - Compile time: nếu biết trước địa chỉ bộ nhớ của chương trình thì có thể gán địa chỉ tuyệt đối lúc biên dịch.

Nhược điểm: phải biên dịch lại nếu thay đổi địa chỉ nạp chương trình.

- Load time: vào thời điểm loading, loader phải chuyển đổi địa chỉ khả tái định vị thành địa chỉ thực dựa trên một địa chỉ nền (base address)

Địa chỉ thực được tính toán vào thời điểm nạp chương trình → phải tiến hành reload nếu địa chỉ thay đổi.

Chuyển đổi địa chỉ (tt)

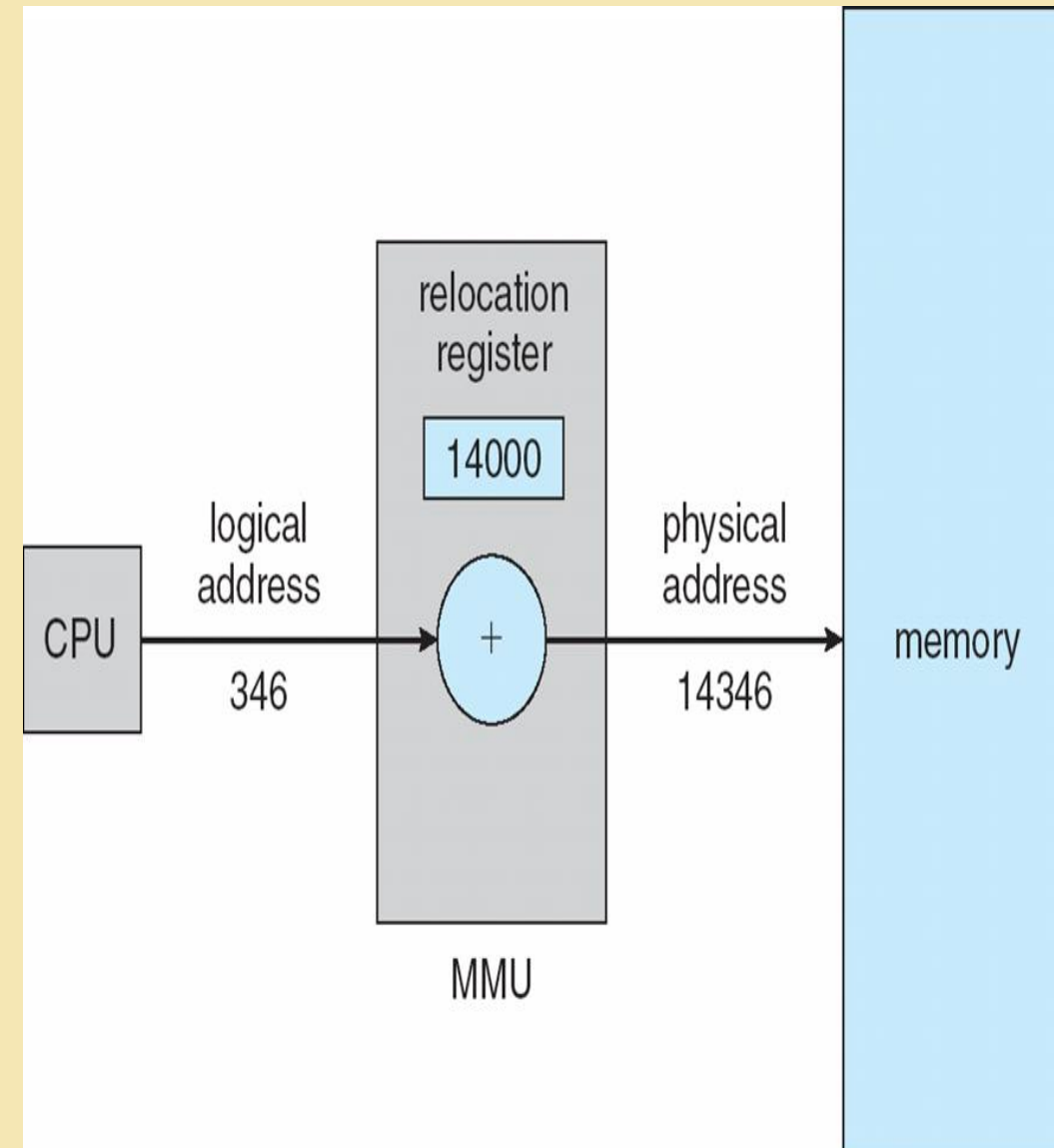
- Execution time: trong quá trình thực thi, tiến trình có thể được di chuyển từ segment này sang segment khác trong bộ nhớ thì quá trình chuyển đổi địa chỉ được trì hoãn đến thời điểm thực thi.
- Cần sự hỗ trợ của phần cứng cho việc ánh xạ địa chỉ.
- Sử dụng trong đa số các OS đa dụng trong đó có các cơ chế swapping, paging, segmentation.

Không gian địa chỉ

- Địa chỉ được khởi tạo bởi CPU – Địa chỉ logic (logical address). Tập hợp địa chỉ logic gọi là không gian địa chỉ logic.
- Địa chỉ nạp vào MAR – địa chỉ vật lý (physical address) . Tập hợp các địa chỉ vật lý gọi là không gian địa chỉ vật lý.
- Compile- time và load – time: địa chỉ logic và physical là xác định.
- Tại thời điểm thực thi: địa chỉ logic thành địa chỉ vật lý, thường gọi là địa chỉ ảo.
- Việc ánh xạ giữa 2 địa chỉ được thực thi bởi Memory Management Unit (MMU).

Tái định vị sử dụng thanh ghi tái định vị (Relocation Register)

- Thủ tục không được nạp cho đến khi nó được gọi
- Sử dụng không gian bộ nhớ tốt hơn; Thủ tục không sử dụng thì không bao giờ được nạp
- Tất cả các thủ tục được giữ trên đĩa theo định dạng nạp có thể tái định vị
- Hữu ích khi số lượng mã lớn cần xử lý những trường hợp xảy ra không thường xuyên
- Không đòi hỏi hỗ trợ đặc biệt từ hệ điều hành
 - Thực thi thông qua thiết kế chương trình
 - Hệ điều hành trợ giúp bằng cách cung cấp thư viện để thực hiện việc nạp động



Mô hình quản lý bộ nhớ

- Một tiến trình phải được nạp hoàn toàn vào bộ nhớ thì mới được thực thi (ngoại trừ khi sử dụng cơ chế overlay).
- Một số cơ chế quản lý bộ nhớ:
 - Phân chia cố định (fixed partitioning)
 - Phân chia động (dynamic partitioning)
 - Phân trang đơn giản (simple paging)
 - Phân đoạn đơn giản (simple segmentation)

Phân mảnh (fragmentation)

- Phân mảnh ngoại: (external fragmentation)- kích thước không gian nhớ còn trống đủ để thỏa mãn một yêu cầu cấp phát, tuy nhiên không gian nhớ này không liên tục → có thể dùng cơ chế **kết nối** (compaction) để gom lại thành vùng nhớ liên tục.
- Phân mảnh nội (internal fragmentation):
 - Kích thước vùng nhớ được cấp phát có thể hơi lớn hơn vùng nhớ yêu cầu.
VD: Cấp phát một khoảng trống 18,464 bytes cho 1 tiến trình yêu cầu 18,462 bytes.
 - Hiện tượng phân mảnh nội thường xảy ra khi bộ nhớ thực được chia thành các khối kích thước cố định (fixed-sized block) và các tiến trình được cấp phát theo đơn vị khối.

Fixed partitioning

- Khi khởi động hệ thống, bộ nhớ chính được chia thành nhiều phần rời nhau gọi là các partition có kích thước bằng nhau hoặc khác nhau.
- Tiến trình nào có kích thước nhỏ hơn hoặc bằng kích thước spartition thì có thể được nạp vào partition đó.
- Nếu chương trình có kích thước lớn hơn partition thì phải dùng cơ chế overlay.
- Nhận xét: Không hiệu quả do bị phân mảnh nội; một chương trình dù lớn hay nhỏ đều được cấp phát trọn 1 partition

Dynamic partitioning

- Số lượng partition không cố định và partition có thể có kích thước khác nhau.
- Mỗi tiến trình được cấp phát chính xác dung lượng bộ nhớ cần thiết.
- Gây ra hiện tượng phân mảnh ngoại.

Chiến lược placement

- Dùng để quyết định cấp phát khối bộ nhớ trống nào cho một tiến trình.
- Mục tiêu: **giảm chi phí compaction**
- Các chiến lược cấp phát placement:
 - Best-fit: chọn khối nhớ trống nhỏ nhất
 - First-fit: chọn khối nhớ trống phù hợp đầu tiên kể từ đầu bộ nhớ
 - Next-fit: chọn khối nhớ trống phù hợp đầu tiên kể từ vị trí cấp phát cuối cùng
 - Worst-fit: chọn khối nhớ trống lớn nhất

Bài tập

Giả sử bộ nhớ chính được phân thành các phân vùng có kích thước là 600K, 500K, 200K, 300K (theo thứ tự), cho biết các tiến trình có kích thước 212K, 417K, 112K và 426K (theo thứ tự) sẽ được cấp phát bộ nhớ như thế nào, nếu sử dụng:

- a) Thuật toán First fit
- b) Thuật toán Best fit
- c) Thuật toán Worst fit
- d) Thuật toán Next fit

Thuật toán nào cho phép sử dụng bộ nhớ hiệu quả nhất trong trường hợp trên.

Cấp phát không liên tục

1. Cơ chế phân trang (paging)

Bộ nhớ vật lý → khung (frame)

- Kích thước của frame là lũy thừa của 2, từ khoảng 512 byte đến 16 MB.

Bộ nhớ luận lý (logical memory) hay **không gian địa chỉ luận lý** là tập mọi địa chỉ luận lý mà một chương trình bất kỳ có thể sinh ra → page

- VD: `mov reg, 1000` // 1000 là 1 địa chỉ luận lý
- **Bảng phân trang (page table)** để ánh xạ địa chỉ luận lý thành địa chỉ thực

Cơ chế phân trang (tt)

page 0
page 1
page 2
page 3

logical
memory

0	1
1	4
2	3
3	7

page table

frame
number

0	
1	page 0
2	
3	page 2
4	page 1
5	
6	
7	page 3

physical
memory

Cơ chế phân trang (tt)

A) Chuyển đổi địa chỉ trong paging

- Địa chỉ luận lý gồm có:
 - + Số hiệu trang (Page number): p
 - + Địa chỉ tương đối trong trang (page offset): d
- Nếu kích thước của không gian địa chỉ luận lý là 2^m Và kích thước của trang là 2^n (đơn vị là byte hay word tùy theo kiến trúc máy) thì

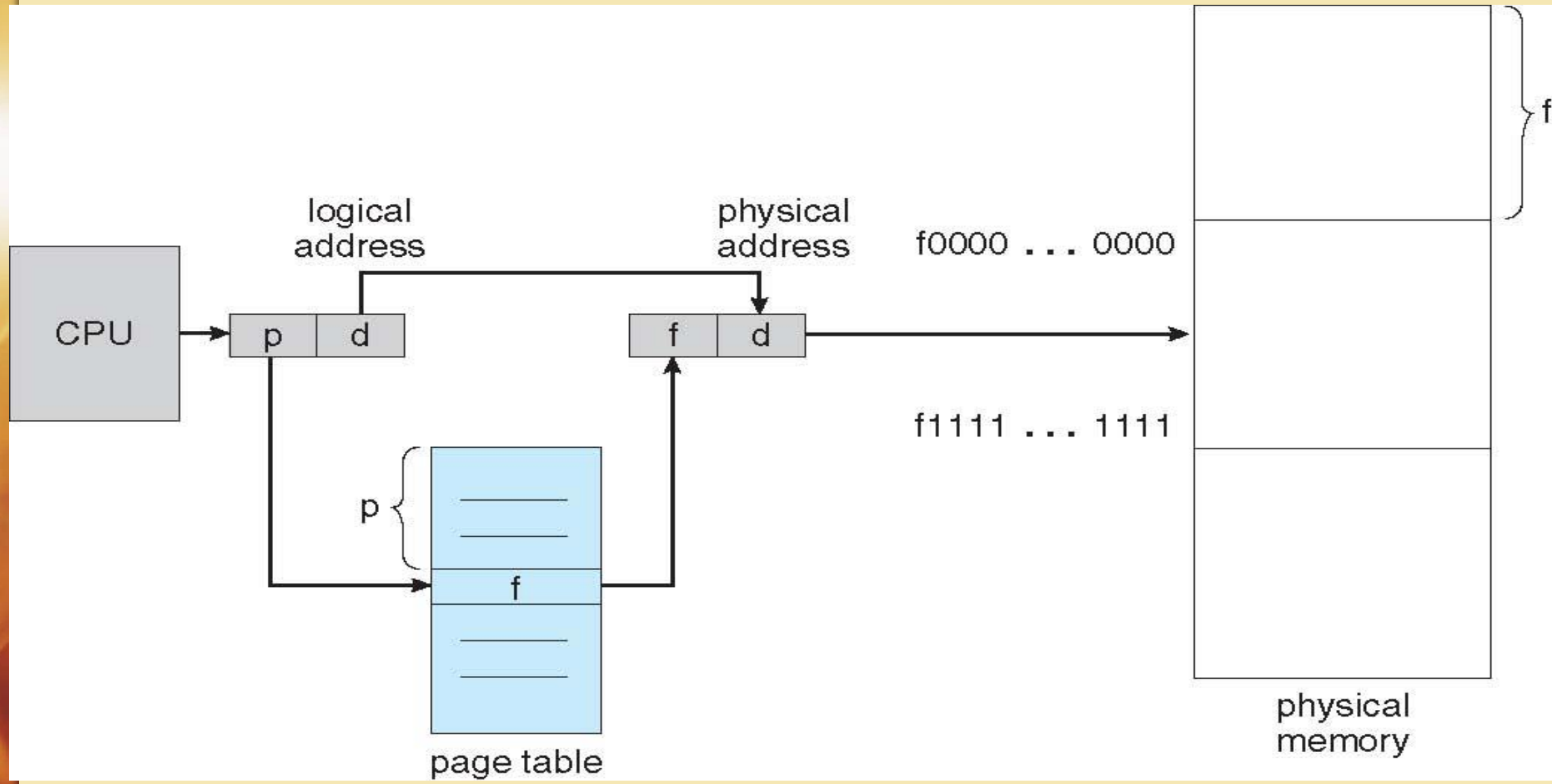
page number	page offset
p	d
$m - n$	n

(định vị từ $0 \rightarrow 2^{m-n} - 1$) (định vị từ $2^n - 1$)

Bảng phân trang sẽ có tổng cộng: $2^m / 2^n = 2^{m-n}$ mục (entry)

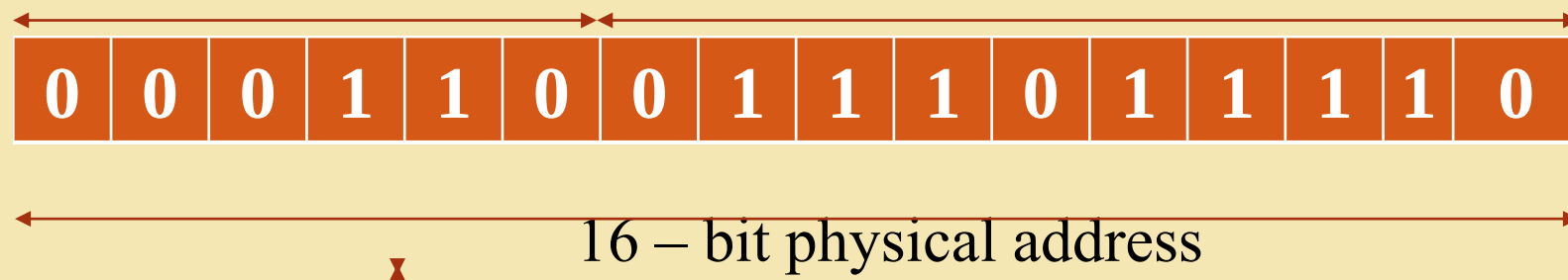
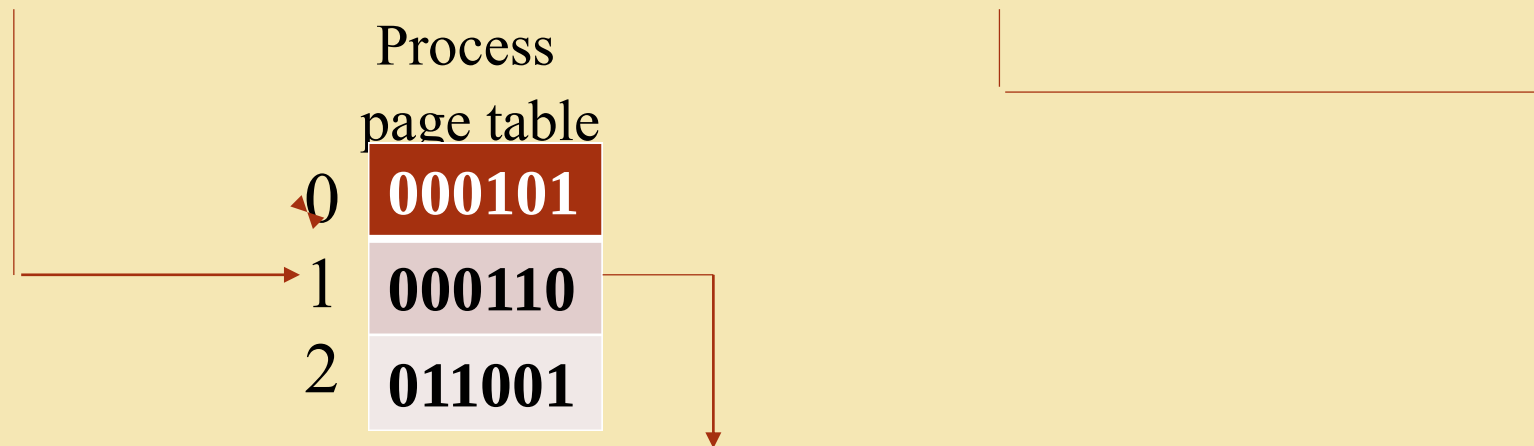
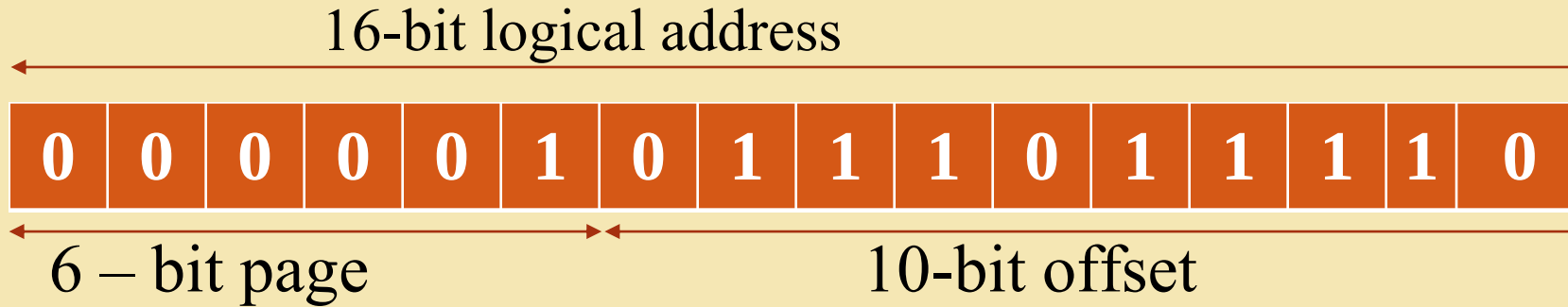
Cơ chế phân trang (tt)

A) Chuyển đổi địa chỉ trong paging



Cơ chế phân trang (tt)

Ví dụ: Chuyển đổi địa chỉ nhớ trong paging



Ví dụ

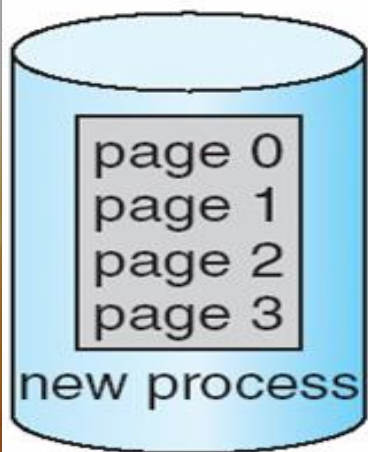
Xét một không gian địa chỉ có 8 trang, mỗi trang có kích thước 1 Kb. Ánh xạ vào bộ nhớ vật lý có 32 khung trang.

- a) Địa chỉ logic gồm bao nhiêu bit ?
- b) Địa chỉ physical gồm bao nhiêu bit ?
- c) Bảng trang có bao nhiêu mục ? Mỗi mục trong bảng trang cần bao nhiêu bit ?

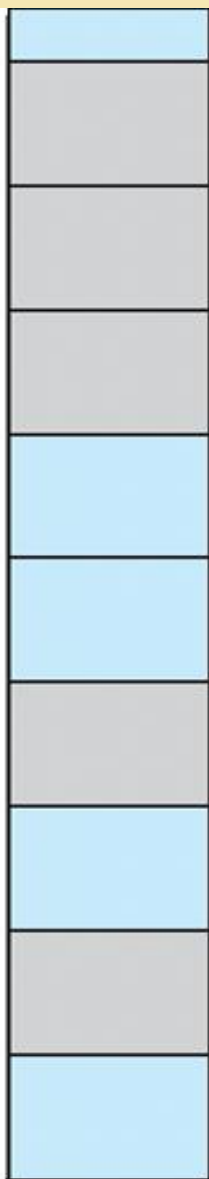
Cơ chế phân trang (tt)

free-frame list

14
13
18
20
15



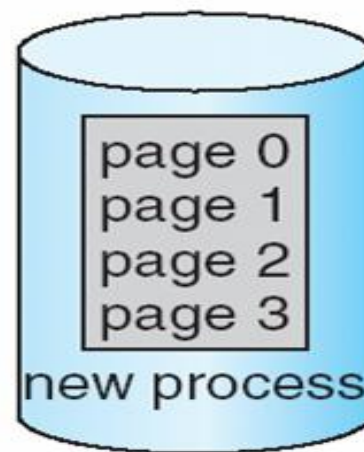
13
14
15
16
17
18
19
20
21



(a)

free-frame list

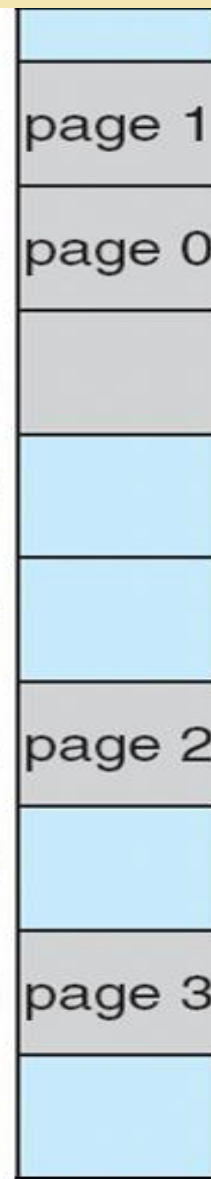
15



0	14
1	13
2	18
3	20

new-process page table

13
14
15
16
17
18
19
20
21



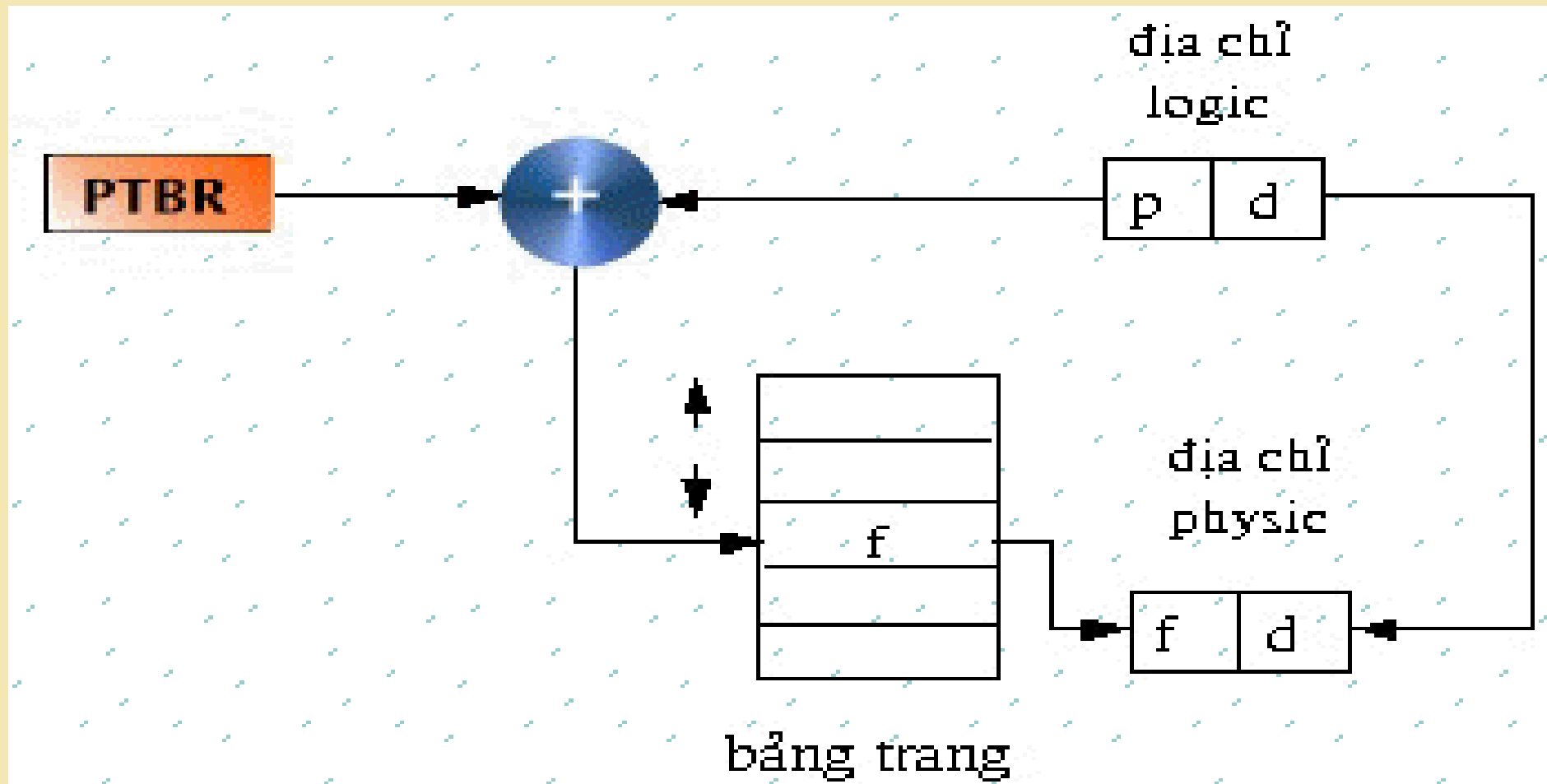
(b)

B) Cài đặt bảng phân trang (Paging hardware)

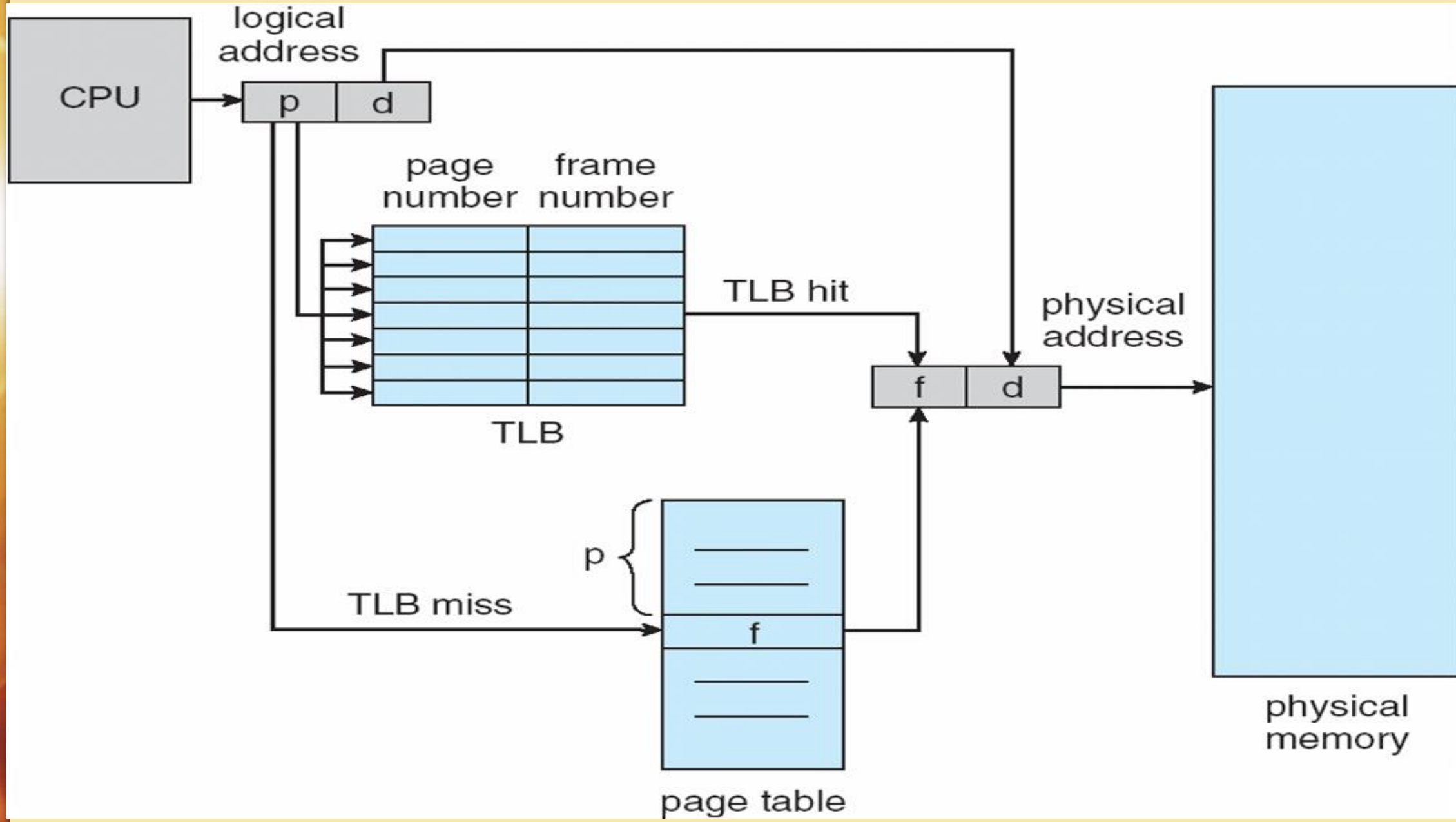
- Bảng phân trang thường được lưu giữ trong bộ nhớ chính.
 - Mỗi tiến trình đều được HĐH cấp một bảng phân trang
 - Thanh ghi **page table base** (PTBR) trỏ đến bảng phân trang
 - Thanh ghi **page table length** (PTLR) biểu thị kích thước của bảng phân trang (có thể được dùng trong cơ chế bảo mật bộ nhớ)
- Thường dùng một bộ phận cache phần cứng có tốc độ truy xuất và tìm kiếm cao, gọi là **thanh ghi kết hợp** (associative register) hoặc **translation lock-aside buffers** (TLBs)

B) Cài đặt bảng trang (Paging hardware)

- Dùng thanh ghi Page-Table Base Register (PTBR)



Paging hardware với TLB



C) Effective access time (EAT)

Tính thời gian truy xuất hiệu dụng (effective access time- EAT)

- Thời gian tìm kiếm trong TLB (associative lookup): ε
- Thời gian một chu kỳ truy xuất bộ nhớ = thời gian tìm kiếm trong bảng phân trang: x
- Hit ratio: tỉ số giữa số lần chỉ số trang được tìm thấy (hit) trong TLB và số lần truy xuất khởi nguồn từ CPU: α
- Thời gian cần thiết để có được chỉ số frame
- + Khi chỉ số trang có trong TLB (hit): $\varepsilon + x$
- + Khi chỉ số trang không có trong TLB (miss): $\varepsilon + x + x$
- Thời gian truy xuất hiệu dụng:

$$\begin{aligned} \text{EAT} &= (\varepsilon + x)\alpha + (\varepsilon + 2x)(1 - \alpha) \\ &= (2 - \alpha)x + \varepsilon \end{aligned}$$

Ví dụ. Tính thời gian truy xuất hiệu dụng

VD1:

- Associative lookup = 20 ns
- Memory access = 100 ns
- Hit ratio = 0,8
- EAT = ?

VD2:

- Associative lookup = 20 ns
- Memory access = 100 ns
- Hit ratio = 0,98
- EAT = ?

Ví dụ 3

Xét một hệ thống sử dụng kỹ thuật phân trang với bảng trang được lưu trữ trong bộ nhớ chính.

- a) Nếu thời gian cho một lần truy xuất bộ nhớ bình thường là 200ns, thì mất bao nhiêu thời gian cho một thao tác truy xuất bộ nhớ trong hệ thống này.
- b) Nếu sử dụng TLBs với hit-ratio (tỉ lệ tìm thấy) là 75% thời gian để tìm trong TLBs xem như bằng 0, tính thời gian truy xuất bộ nhớ trong hệ thống (effective memory reference time)

D) Tổ chức bảng trang – Phân trang đa cấp

Các hệ thống hiện đại đều hỗ trợ không gian địa chỉ ảo rất lớn (2^{32} đến 2^{64}), ở đây giả sử là 2^{32}

- Giả sử kích thước trang nhớ là 4KB ($= 2^{12}$)

- bảng phân trang sẽ có $2^{32}/2^{12} = 2^{20} = 1\text{M}$ mục

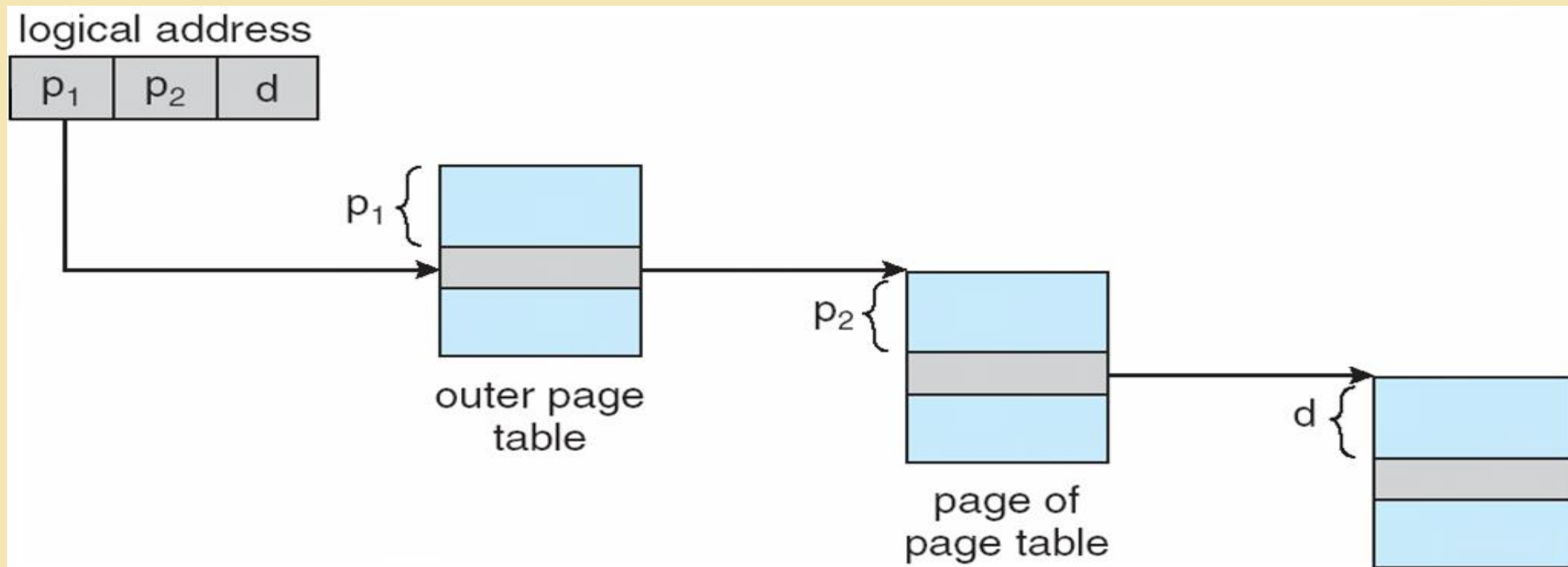
- Giả sử mỗi mục gồm 4 byte thì mỗi process cần 4 MB cho bảng phân trang

+ VD: Phân trang hai cấp

page number		page offset
p_1	p_2	d
10	10	12

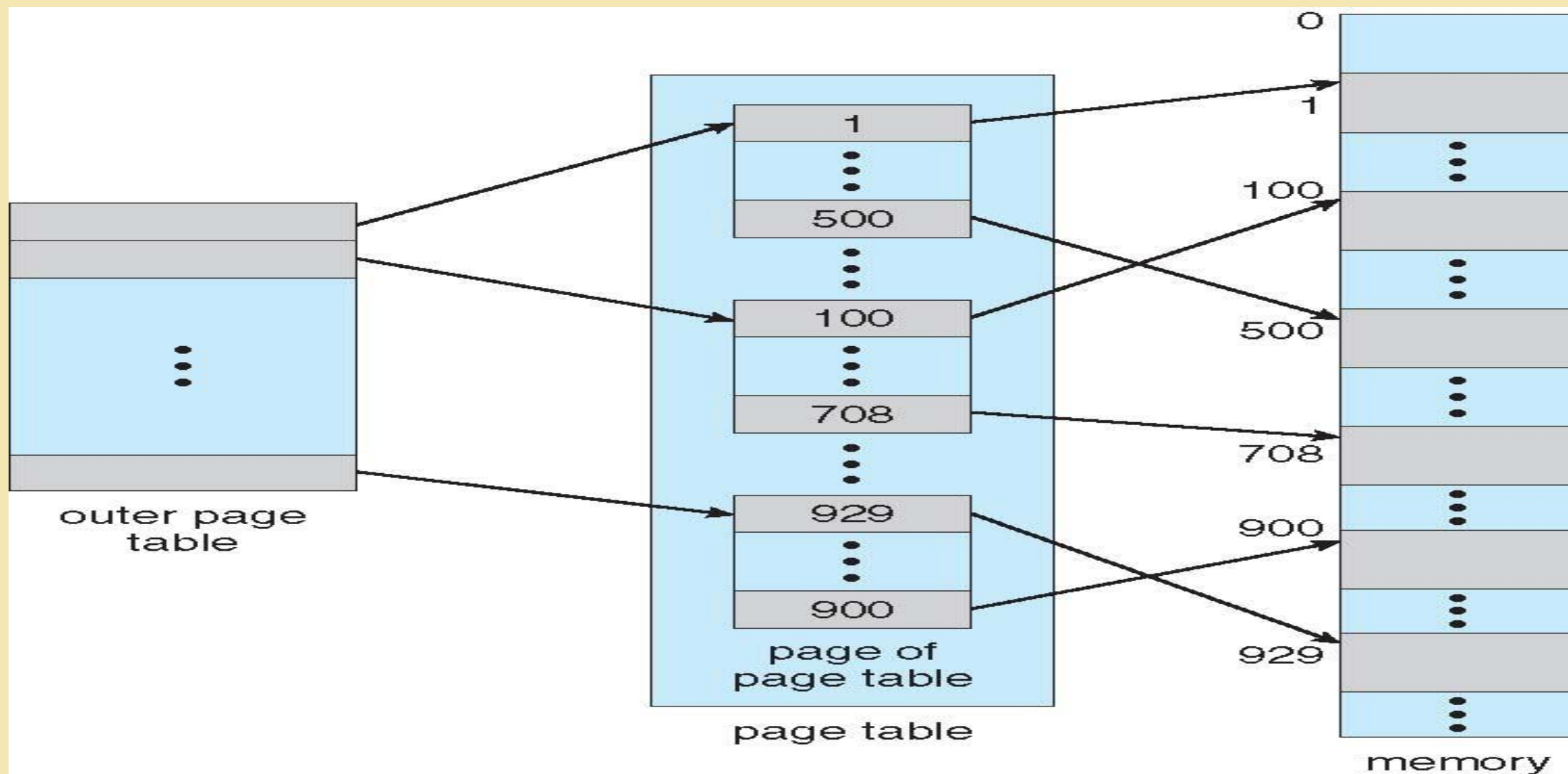
Phân trang đa cấp (tt)

- Sơ đồ chuyển đổi địa chỉ (address-translation scheme) cho cơ chế bảng phân trang 2 mức, 32-bit địa chỉ



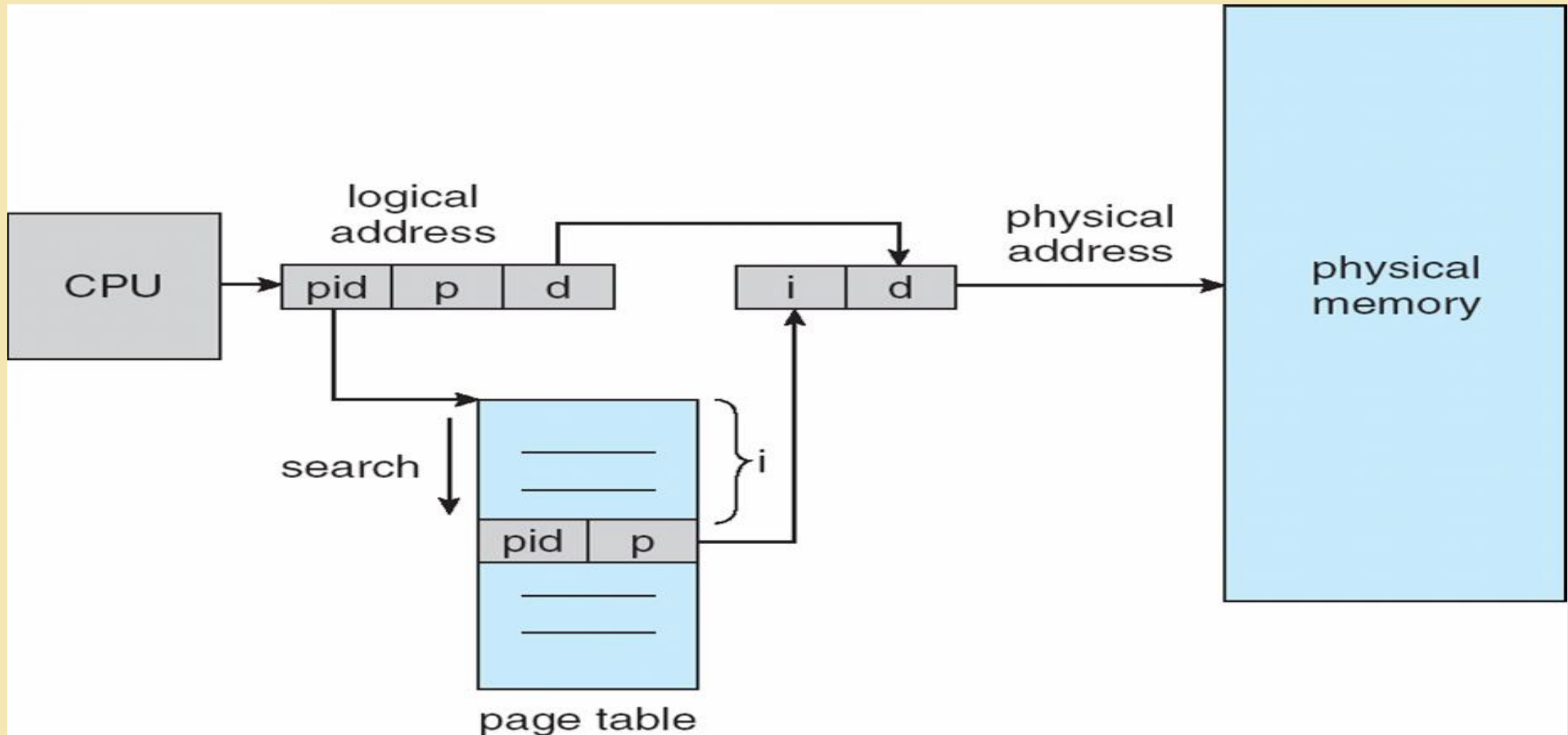
Tổ chức bảng trang

VD: Phân trang đa cấp



D) Tổ chức bảng trang

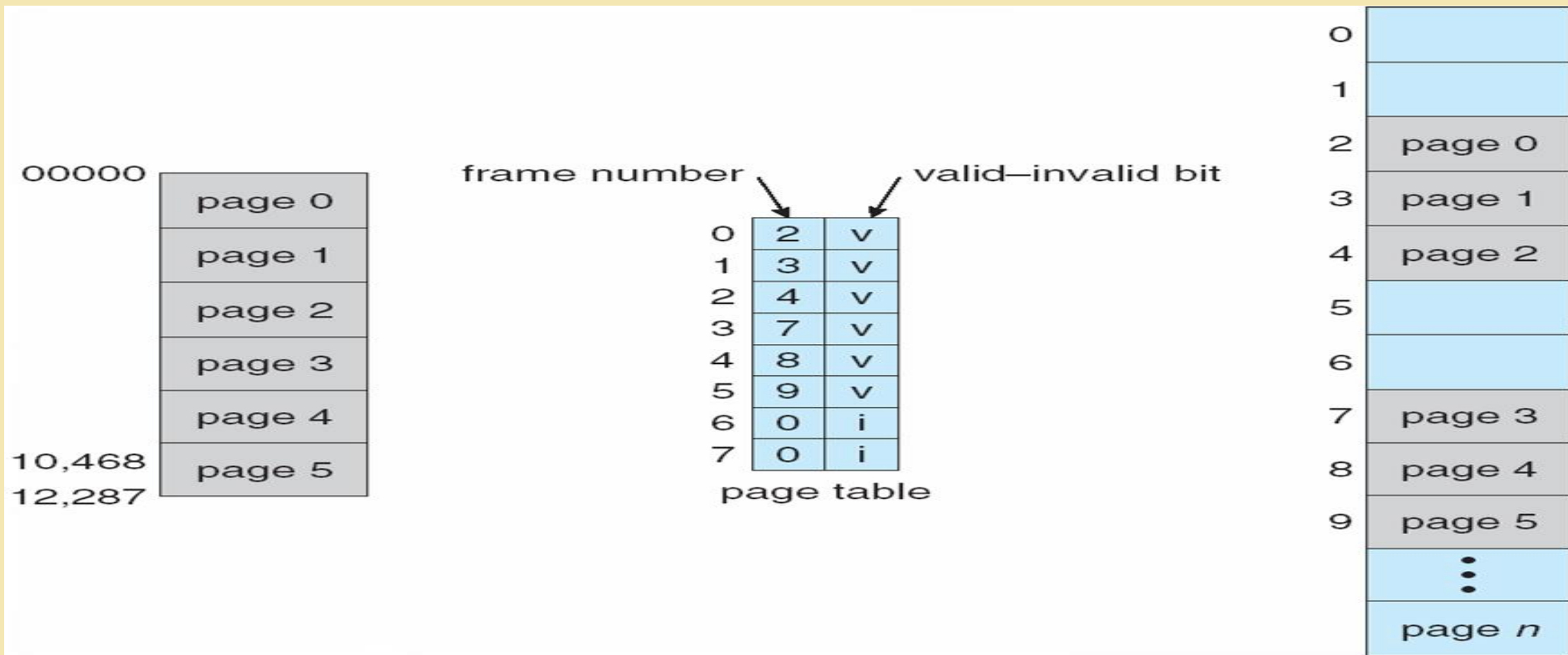
- Bảng trang nghịch đảo: sử dụng cho tất cả các process (PID,p,d)



E) Bảo vệ bộ nhớ

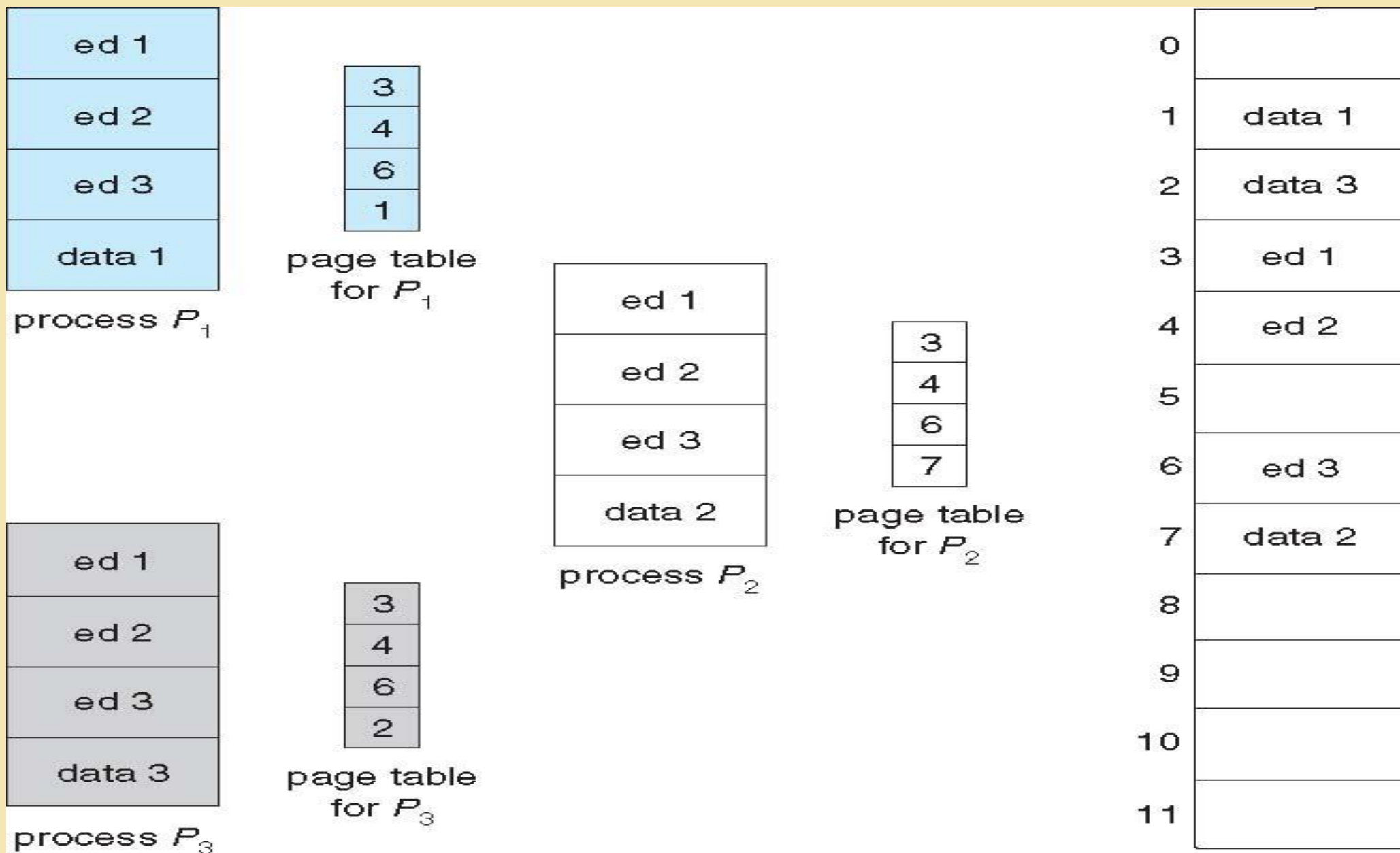
- Việc bảo vệ bộ nhớ được thực hiện bằng cách gắn với frame các bit bảo vệ (protection bits) được giữ trong bảng phân trang. Các bit này biểu thị các thuộc tính sau:
 - + read-only, read-write, execute-only
- Ngoài ra còn có một valid/invalid bit gắn với mỗi mục trong bảng phân trang
 - + valid: cho biết là trang của process, do đó là một trang hợp lệ
 - + invalid: cho biết là trang không của process, do đó là một trang bất hợp lệ

Bảo vệ bằng valid/invalid bit



- Mỗi trang nhớ có kích thước $2K = 2048$
- Process có kích thước 10,468 → phân mảnh nội ở frame 9 (chứa page 5), các địa chỉ ảo > 12287 là các địa chỉ invalid
- Dùng PTLR để kiểm tra truy xuất đến bảng phân trang có nằm trong bảng hay không

F) Chia sẻ trang nhớ

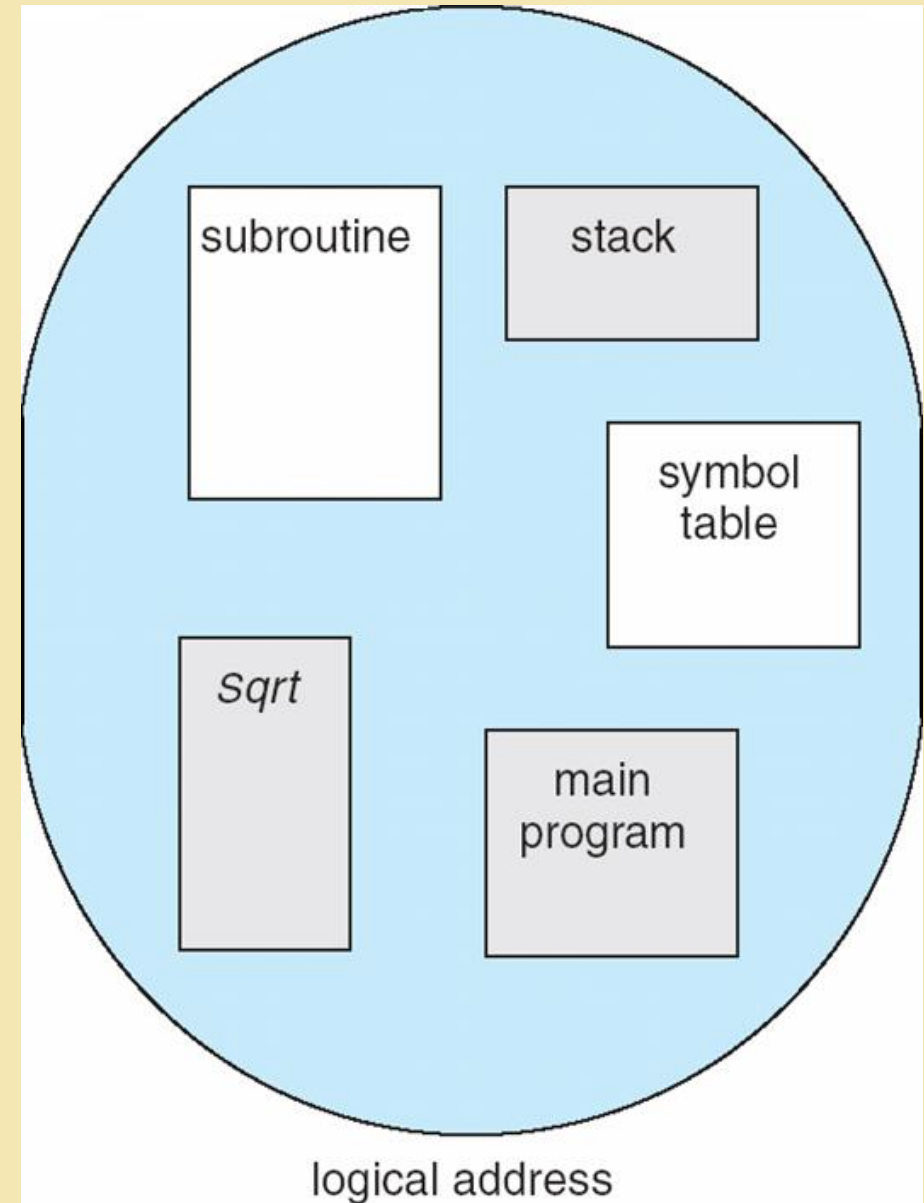


2. Phân đoạn (segmenttation)

- Nhìn lại cơ chế phân trang
 - User view (không gian địa chỉ ảo) tách biệt với không gian bộ nhớ thực. Cơ chế phân trang thực hiện phép ánh xạ user-view vào bộ nhớ thực
- Trong thực tế, dưới góc nhìn của user, một chương trình cấu thành từ nhiều đoạn (segment). Mỗi đoạn là một đơn vị luận lý của chương trình như:
 - Main program, procedure, function
 - Local variables, global variables, common block, stack, symbol table, arrays...

User view của một chương trình

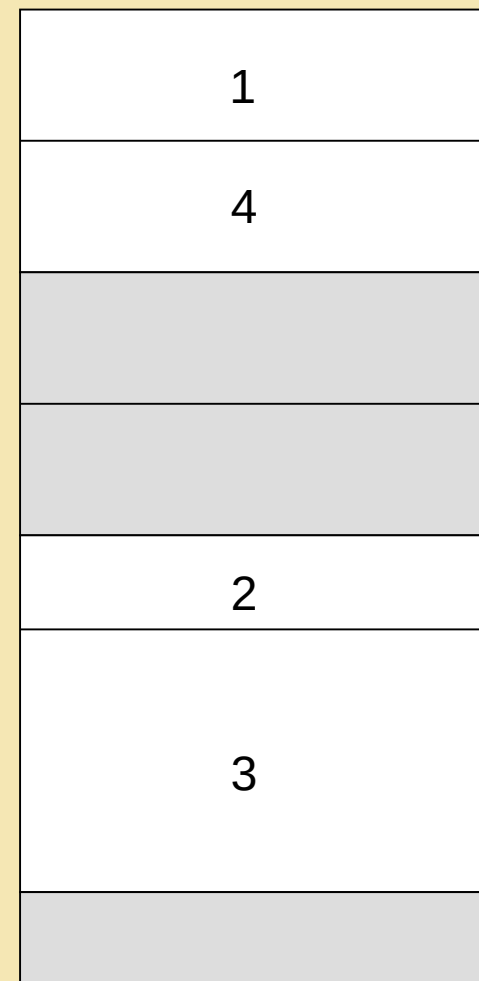
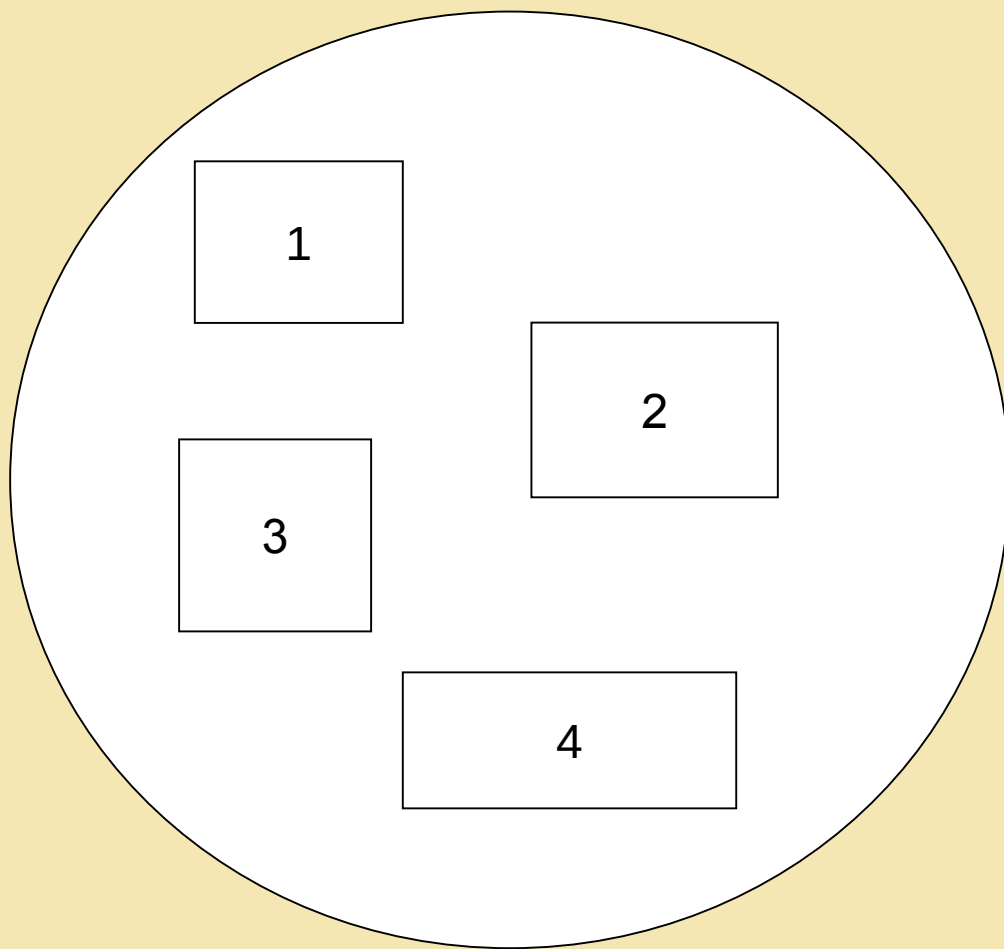
- Thông thường, một chương trình được biên dịch. **Trình biên dịch sẽ tự động xây dựng các segment.**
- VD: trình biên dịch Pascal sẽ tạo ra các segment sau:
 - Global variables
 - Procedure call stack
 - Procedure/function code
 - Local variable
- Trình loader sẽ gán mỗi segment một số định danh riêng.



Phân đoạn

- Dùng cơ chế phân đoạn để quản lý bộ nhớ có hỗ trợ user view
 - Không gian địa chỉ ảo là một tập các đoạn, mỗi đoạn có tên và kích thước riêng
 - Một địa chỉ luận lý được định vị bằng tên đoạn và độ dời (offset) bên trong đoạn đó (so sánh với phân trang)

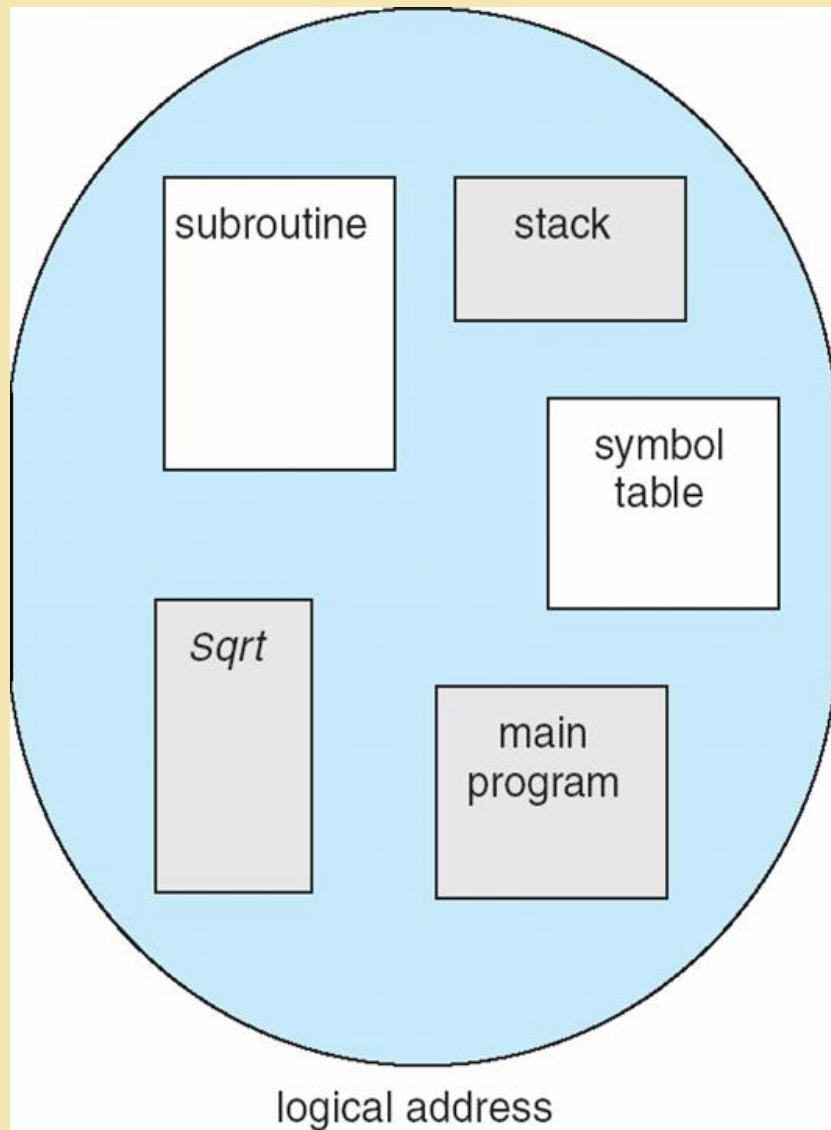
Phân đoạn



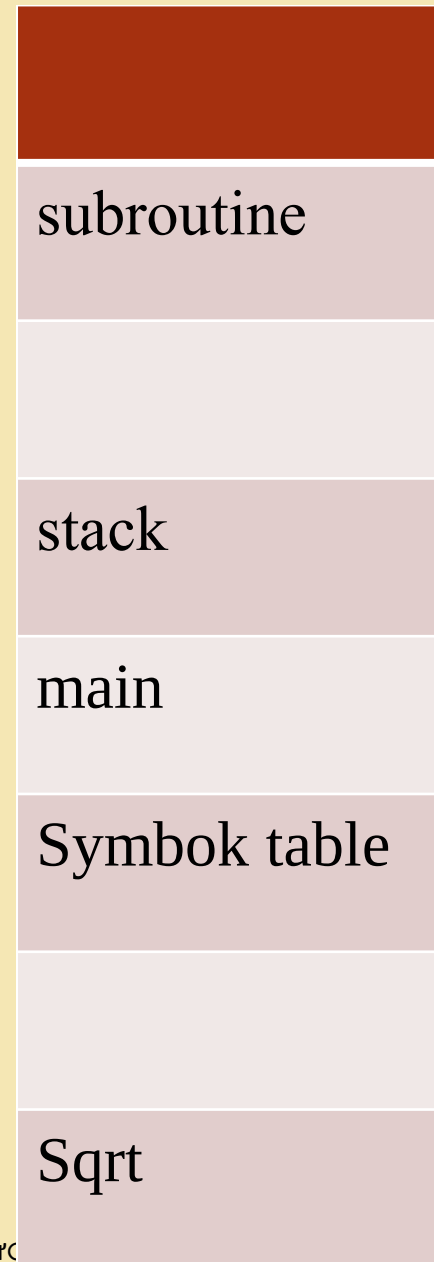
Cài đặt phân đoạn

- Địa chỉ luận lý : là một cặp giá trị
(segment number, offset)
 - Bảng phân đoạn (segment table): gồm nhiều mục, mỗi mục chứa :
 - base: chứa địa chỉ khởi đầu của segment trong bộ nhớ
 - Limit: xác định kích thước của segment
 - Segment-table base register(STBR): trỏ đến vị trí bảng phân đoạn trong bộ nhớ
 - Segment-table length register(STLR): số lượng segment của chương trình
- Một chỉ số segment s là hợp lệ nếu $s < \text{STLR}$

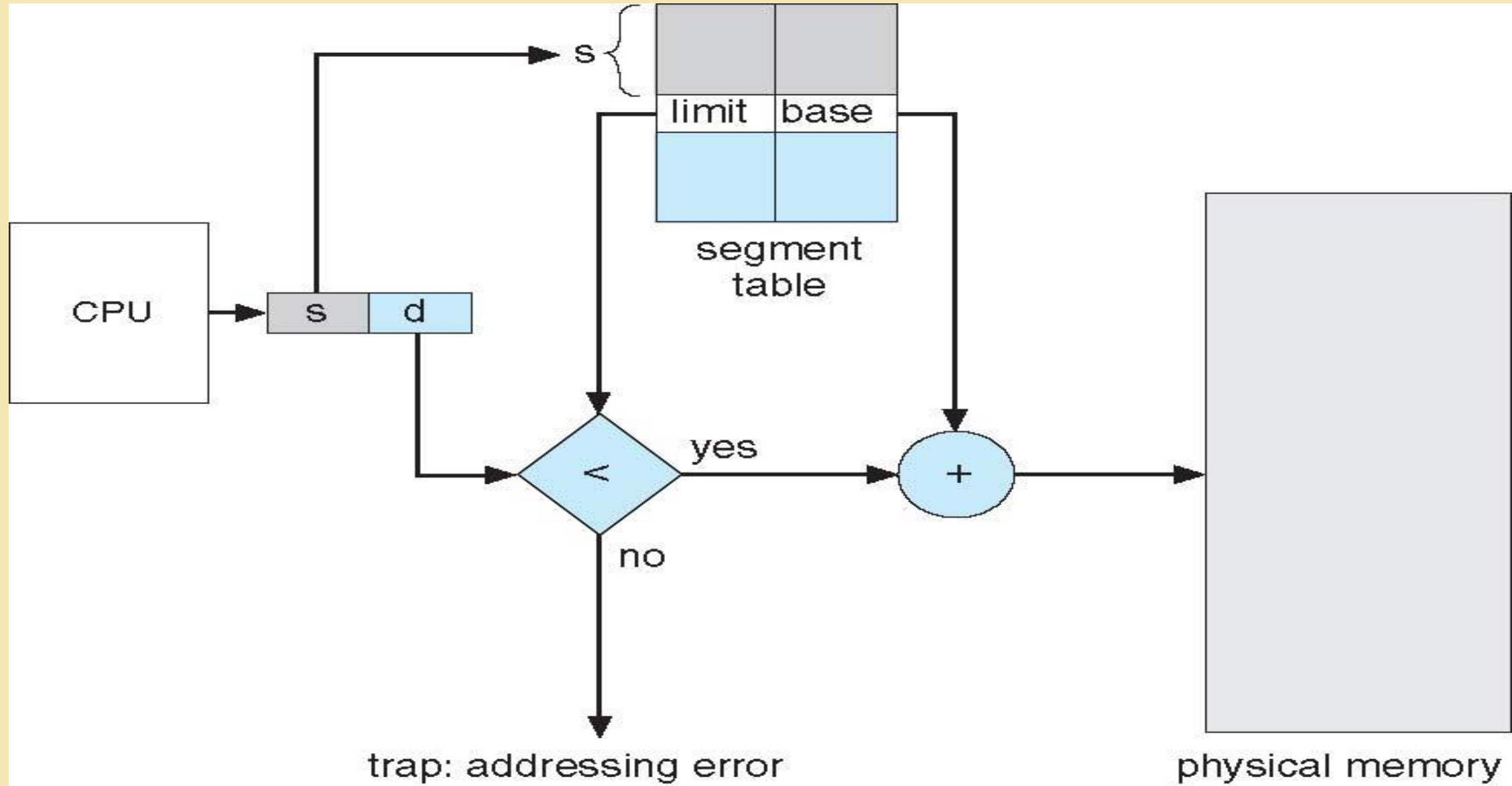
Ví dụ



Limit	Base
1000	1400
400	6300
400	4300
1100	3200
1000	4700



Phần cứng hỗ trợ phân đoạn



Ví dụ

- Logical Address: 0001 001011110000

0	001011101110	0000010000000000
1	011110011110	00100000000100000

3. Kết hợp phân trang và phân đoạn

- Kết hợp phân trang và phân đoạn nhằm kết hợp các ưu điểm đồng thời hạn chế các khuyết điểm của phân trang và phân đoạn:
 - Vấn đề của phân đoạn: nếu một đoạn quá lớn thì có thể không nạp được nó vào bộ nhớ
 - Ý tưởng giải quyết: paging đoạn, khi đó chỉ cần giữa trong bộ nhớ các page của đoạn hiện đang cần
 - **Logic Addr = $\langle s, p, d \rangle$**



Hết chương

Câu 1.

Xét một hệ thống sử dụng kỹ thuật phân trang với bảng trang được lưu trữ trong bộ nhớ chính. Nếu thời gian cho 1 lần truy xuất bộ nhớ bình thường là 300 ns thì mất bao nhiêu thời gian cho một thao tác truy xuất bộ nhớ trong hệ thống này.

- a. 300 ns
- b. 500 ns
- c. 600 ns
- d. Cả 3 đều sai

Câu 2.

Xét một hệ thống sử dụng kỹ thuật phân trang với bảng trang được lưu trữ trong bộ nhớ chính. Thời gian cho một lần truy xuất bộ nhớ bình thường là 300 ns. Nếu sử dụng TLBs với hit-ratio là 75%, thời gian để tìm trong TLBs coi như bằng 0. Tính thời gian truy xuất bộ nhớ trong hệ thống.

- a. 200 ns
- b. 550 ns
- c. 600 ns
- d. Cả 3 đều sai

Câu 3

Xét một hệ thống sử dụng kỹ thuật phân trang với bảng trang được lưu trữ trong bộ nhớ chính. Thời gian cho một lần truy xuất bộ nhớ bình thường là 200 ns. Nếu sử dụng TLBs với hit-ratio là 85%, thời gian để tìm trong TLBs coi như bằng 0. Tính thời gian truy xuất bộ nhớ trong hệ thống.

- a. 200 ns
- b. 400ns
- c. 370 ns
- d. 220 ns

Câu 4

Tiến trình luôn ở trạng thái running

- a. Tiến trình hướng I/O
- b. Tiến trình hướng CPU
- c. Cả a và b đều sai
- d. Cả a và b đều đúng

Câu 5

Địa chỉ vật lý 4100 sẽ được chuyển thành địa chỉ ảo bao nhiêu. Biết kích thước mỗi frame là 1Kb

- a. 4100
- b. 1028
- c. 5124
- d. Cả 3 đều sai

Page table

6
4
5
7
1
9